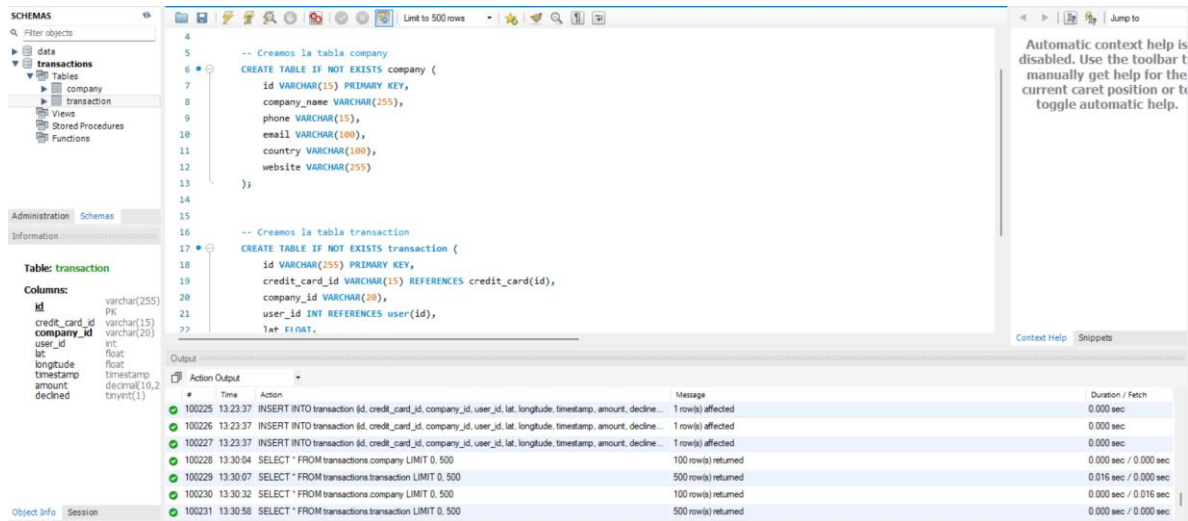


NIVEL 1

Ejercicio 1: A partir de los documentos adjuntos (estructura_datos y datos_introducir), importa las dos tablas.



Muestra las principales características del esquema creado y explica las diferentes tablas y variables que existen.

El esquema consta de dos tablas principales conectadas a través de una relación relacional:

Tabla company: En esta tabla se almacena la información de varias empresas o comercios.

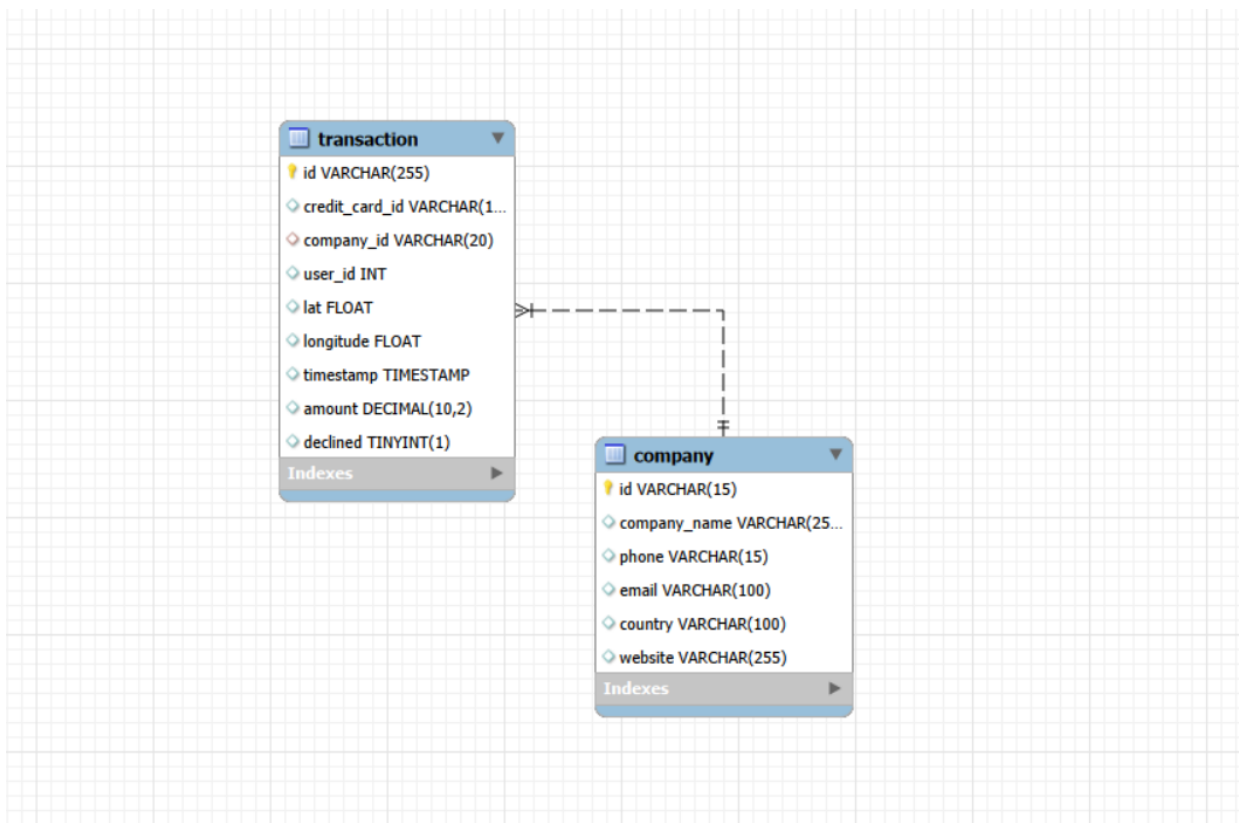
- **id:** Clave primaria (Primary Key) esta identifica de forma única a cada empresa.
- **company_name:** Nombre de la empresa.
- **phone:** Teléfono de contacto.
- **email:** Correo electrónico de contacto.
- **country:** País donde está ubicada la empresa.
- **website:** URL de la página web oficial.

B. Tabla transaction: En esta tabla registran los movimientos financieros o transacciones comerciales realizadas.

- **id:** Clave primaria (Primary Key) que permite identificar individualmente cada transacción.
- **credit_card_id :** Identificador de la tarjeta de crédito.
- **company_id:** Clave foránea (Foreign Key) que vincula la transacción con la empresa correspondiente en la tabla company.

- **user_id:** Identificador del usuario que realiza la compra.
- **lat:** Latitud de las coordenadas geográficas donde se efectuó la operación.
- **longitude:** Longitud geográfica de la operación.
- **timestamp:** Fecha y hora exacta en la que se procesó el movimiento.
- **amount:** El importe de la transacción.
- **declined:** indica si la transacción fue declinada o rechazada (1) o si fue aprobada con éxito (0).

Asegúrate de incluir un diagrama que ilustre la relación entre las distintas tablas y variables.



Existe una relación de **Uno a Muchos (1:N)** mediante `company_id` de la tabla `transaction`, el cual actúa como clave foránea y hace referencia directa a la clave primaria `id` de la tabla `company`, en esta relación se establece que, una empresa puede estar asociada a múltiples transacciones registradas, pero cada transacción pertenece exclusivamente a una única empresa.

Ejercicio 2: Utilizando JOIN realizarás las siguientes consultas:

- Listado de los países que están generando ventas.

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Schemas' tree with 'data' and 'transactions' schemas. The 'transactions' schema is expanded, showing 'Tables', 'company', 'transaction', 'Views', 'Stored Procedures', and 'Functions'. The 'company' table is selected. The right pane shows the query editor with the following SQL code:

```
16
17 #Listado de los países que están generando ventas.
18 SELECT c.country, COUNT(t.amount) AS ventas_generadas
19 FROM company c
20 JOIN transaction t
21 ON c.id = t.company_id
22 GROUP BY c.country
23 ORDER BY ventas_generadas;
```

The 'Result Grid' shows the following data:

country	ventas_generadas
China	916
Spain	1515
Canada	2107
Australia	2562
New Zealand	2641
Ireland	2659
Norway	3030
Belgium	3531
United States	4036
France	4700

The 'Output' pane shows the execution message: '1 16:16:03 SELECT c.country, COUNT(t.amount) AS ventas_generadas FROM company c JOIN transaction t ON c.id = t.c... 15 row(s) returned'. The duration is 0.297 sec / 0.000 sec.

- Desde cuántos países se generan las ventas.

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Schemas' tree with 'data' and 'transactions' schemas. The 'transactions' schema is expanded, showing 'Tables', 'company', 'transaction', 'Views', 'Stored Procedures', and 'Functions'. The 'company' table is selected. The right pane shows the query editor with the following SQL code:

```
25
26
27 #Desde cuántos países se generan las ventas.
28 SELECT COUNT(DISTINCT c.country) AS cant_paises
29 FROM company c
30 JOIN transaction t
31 ON c.id = t.company_id;
```

The 'Result Grid' shows the following data:

cant_paises
15

The 'Output' pane shows the execution message: '1 16:17:23 SELECT COUNT(DISTINCT c.country) AS cant_paises FROM company c JOIN transaction t ON c.id = t.c... 1 row(s) returned'. The duration is 0.079 sec / 0.000 sec.

- Identifica a la compañía con la mayor media de ventas.

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Schemas' tree with 'data' and 'transactions' schemas. The 'transactions' schema is expanded, showing 'Tables', 'company', 'transaction', 'Views', 'Stored Procedures', and 'Functions'. The 'company' table is selected. The right pane shows the query editor with the following SQL code:

```
29
30
31 #Identifica a la compañía con la mayor media de ventas
32 SELECT c.id, c.company_name, ROUND(AVG(t.amount)) AS med_ventas
33 FROM company c
34 JOIN transaction t
35 ON c.id = t.company_id
36 GROUP BY c.id, c.company_name
37 ORDER BY med_ventas DESC
38 LIMIT 1;
```

The 'Result Grid' shows the following data:

id	company_name	med_ventas
b-2222	Ac Fermentum Incorporated	285

The 'Output' pane shows the execution message: '1 10:37:36 SELECT c.id, c.company_name, ROUND(AVG(t.amount)) AS med_ventas FROM company c JOIN transaction t ... 1 row(s) returned'. The duration is 0.485 sec / 0.000 sec.

Ejercicio 3: Utilizando sólo subconsultas (sin utilizar JOIN):

- Muestra todas las transacciones realizadas por empresas de Alemania.

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Schemas' tree with 'data' expanded, showing 'company', 'transaction', 'Views', 'Stored Procedures', and 'Functions'. The 'Schema: transactions' is selected. The central pane shows a SQL query:

```
35
36 /*Ejercicio 3
37 Utilizando sólo subconsultas (sin utilizar JOIN): */
38 #Muestra todas las transacciones realizadas por empresas de Alemania.
39 SELECT *,
40 (SELECT DISTINCT country
41 FROM company
42 WHERE country LIKE "Germany") AS pais
43 FROM transaction t
44 WHERE EXISTS
45 (SELECT id
46 FROM company c
47 WHERE t.company_id = c.id
48 AND country = "Germany")
49 ORDER BY amount DESC;
```

The 'Result Grid' shows the following data:

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined	pais
FEABE7D9-E43A-47CD-9861-4DBA277F7180	CcS-4997	b-2358	435	52.2288	19.3977	2018-06-27 00:02:36	811.46	0	Germany
04FA31CF-74E2-44F2-83C6-92564EA458F0	CcS-5025	b-2614	444	51.5723	10.6503	2024-01-27 03:24:30	800.45	0	Germany
45CE6278-6A2C-4451-4E58-516800C472A7	CcS-5053	b-2550	472	52.5348	5.52788	2017-03-20 09:45:25	727.55	0	Germany
20B2C19B-2B29-419D-8E51-9E4C60EDC122	CcS-4880	b-2358	299	46.4966	1.54041	2018-01-22 19:24:08	725.84	0	Germany
DF4228FF-5458-4F29-9769-708000942F1	CcS-5036	b-2566	455	51.8561	19.6304	2024-07-24 09:39:02	724.44	0	Germany
ASD0D751B4-4601-96CF-83544DCB7F0D	CcS-6481	b-2614	1900	41.1605	-8.62746	2018-06-06 02:06:26	718.20	0	Germany
D8B6D0D0-4DE1-48C6-8AC9-7FC6B498B8E	CcS-9524	b-2302	4943	-42.272	12.3065	2017-09-08 10:21:47	718.52	0	Germany

The 'Output' pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	10:40:25	SELECT *	(SELECT DISTINCT country FROM company WHERE country LIKE "Germany") AS pais FROM tra... 500 row(s) returned	0.062 sec / 0.000 sec

- Lista las empresas que han realizado transacciones por un amount superior a la media de todas las transacciones.

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Schemas' tree with 'data' expanded, showing 'company', 'transaction', 'Views', 'Stored Procedures', and 'Functions'. The 'Schema: transactions' is selected. The central pane shows a SQL query:

```
53
54
55 #Lista las empresas que han realizado transacciones por un amount superior a la media de todas las transacciones.
56 SELECT company_name
57 FROM company c
58 WHERE EXISTS (
59 SELECT company_id
60 FROM transaction t
61 WHERE c.id = t.company_id
62 AND amount > (SELECT AVG(amount) FROM transaction)
63 )
64
```

The 'Result Grid' shows the following data:

company_name
Ac Fermentum Incorporated
Magna A Neque Industries
Fusce Corp.
Convalis In Incorporated
Anite Incula Nec Foundation
Donec Ltd
Sed Nunc Ltd
Amet Nulla Donec Corporation

The 'Output' pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	10:44:17	SELECT company_name	FROM company c WHERE EXISTS (SELECT company_id FROM transaction t WHE... 100 row(s) returned	0.109 sec / 0.000 sec

- **Eliminarán del sistema las empresas que carecen de transacciones registradas, entrega el listado de estas empresas.**

The screenshot shows a SQL IDE interface. On the left, the 'SCHEMAS' pane shows a tree view with 'data' and 'transactions' schemas. The 'transactions' schema is expanded, showing 'company' and 'transaction' tables. The main editor displays a SQL query:

```
73
74 /* Eliminarán del sistema las empresas que carecen de transacciones registradas, entrega el listado de estas empresas.
75 * SELECT company_name
76 * FROM company c
77 * WHERE NOT EXISTS (
78 *   SELECT DISTINCT company_id
79 *   FROM transaction t
80 *   WHERE c.id = t.company_id
81 *   AND company_id IS NOT NULL
82 * )
83 */
```

The 'Result Grid' shows a single column 'company_name'. The 'Output' pane shows a message: '1 10:47:02 SELECT company_name FROM company c WHERE NOT EXISTS (SELECT DISTINCT company_id FROM tr... 0 row(s) returned'.

Ejercicio 4:

Tu tarea es diseñar y crear una tabla llamada "credit_card" que almacene detalles cruciales sobre las tarjetas de crédito. La nueva tabla debe ser capaz de identificar de forma única cada tarjeta

The screenshot shows a SQL IDE interface. On the left, the 'SCHEMAS' pane shows a tree view with 'data' and 'transactions' schemas. The 'transactions' schema is expanded, showing 'company' and 'transaction' tables. The main editor displays a SQL query:

```
73
74 /* Ejercicio 4
75 * Tu tarea es diseñar y crear una tabla llamada "credit_card" que almacene detalles cruciales sobre las tarjetas de crédito.
76 * La nueva tabla debe ser capaz de identificar de forma única cada tarjeta */
77 * CREATE TABLE credit_card (
78 *   id VARCHAR(100) PRIMARY KEY,
79 *   iban VARCHAR(250) NOT NULL,
80 *   pan VARCHAR(250) NOT NULL,
81 *   pin VARCHAR(250) NOT NULL,
82 *   cvv VARCHAR(250) NOT NULL,
83 *   expiring_date VARCHAR(250) NOT NULL
84 * )
85 */
86
```

The 'Table: credit_card' pane shows the table structure:

Columns:	data type
id	varchar(100) PK
iban	varchar(250)
pan	varchar(250)
pin	varchar(250)
cvv	varchar(250)
expiring_date	varchar(250)

The 'Output' pane shows a message: '1 10:49:22 CREATE TABLE credit_card (id VARCHAR(100) PRIMARY KEY, iban VARCHAR(250) NOT NULL, pan VARCHAR... 0 row(s) affected'.

Después de crear la tabla será necesario que ingreses la información del documento denominado "datos_introducir_credit".

Automatic context help is disabled. Use the toolbar manually get help for the current caret position or to toggle automatic help.

id	iban	pan	pin	cvv	expiring_date
CcS-4857	XX4857591835292505850771	2314242385113924	1819	467	09/27/25
CcS-4858	XX8581768137002436094025	6582720299715533	3964	817	12/28/28
CcS-4859	XX78263049142353609370	8861684536289942	4983	277	11/26/26
CcS-4860	XX55599036833304645299	248115551548459	6876	661	07/27/27
CcS-4861	XX2035182877195191627307	1308930301149557	5710	398	04/25/26

#	Time	Action	Message	Duration / Fetch
4997	16:38:49	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES (CcS-9578, 'XX991539646456...	1 row(s) affected	0.000 sec
4998	16:38:49	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES (CcS-9579, 'XX296393091587...	1 row(s) affected	0.000 sec
4999	16:38:49	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES (CcS-9580, 'XX781258889851...	1 row(s) affected	0.000 sec
5000	16:38:49	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES (CcS-9581, 'XX915670516405...	1 row(s) affected	0.000 sec
5001	16:38:56	SELECT * FROM transactions.credit_card LIMIT 0, 500	500 row(s) returned	0.000 sec / 0.000 sec

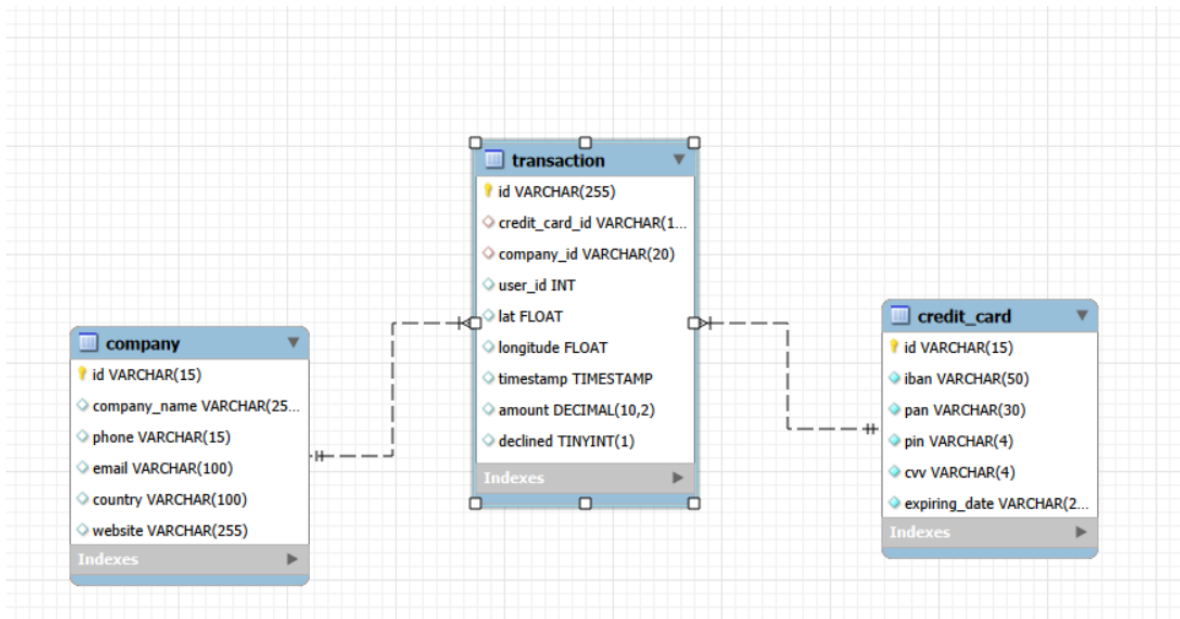
y establecer una relación adecuada con las otras dos tablas ("transaction" y "company").

Automatic context help is disabled. Use the toolbar manually get help for the current caret position or to toggle automatic help.

```

101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
244
```

Recuerda mostrar el diagrama y realizar una breve descripción del mismo



Es una base de datos formada por tres tablas principales: **company**, **transaction** y **credit_card**, y se componen de dos relaciones de tipo **uno a muchos (1:N)**.

La primera se establece entre **company** y **transaction**, ya que una empresa puede estar asociada a múltiples transacciones, mientras que cada transacción corresponde a una sola empresa.

La segunda relación se da entre **credit_card** y **transaction**, ya que una tarjeta de crédito puede intervenir en varias transacciones, pero cada transacción queda vinculada a una única tarjeta.

Ejercicio 5

El departamento de Recursos Humanos ha identificado un error en el número de cuenta asociado a su tarjeta de crédito con ID CcU-2938. La información que debe mostrarse para este registro es: TR323456312213576817699999. Recuerda mostrar que el cambio se realizó.

Verificamos que exista el ID CcU-2938 y que el registro es incorrecto en la tabla credit_card

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Schemas' tree with 'transactions' expanded, showing tables like 'company', 'credit_card', and 'transaction'. The central pane contains a SQL query:

```
96
97 # Verificamos que exista el ID CcU-2938 y que el registro es incorrecto en la tabla credit_card
98 SELECT id, iban
99 FROM credit_card
100 WHERE id = 'CcU-2938';
101
```

The 'Result Grid' shows one row of data:

id	iban
CcU-2938	TR301950312213576817638661

The bottom pane shows the 'Output' window with the following message:

#	Time	Action	Message	Duration / Fetch
1	10:58:54	SELECT id, iban FROM credit_card WHERE id = 'CcU-2938' LIMIT 0.500	1 row(s) returned	0.000 sec / 0.000 sec

Realización del cambio solicitado y realizamos una verificación de que el cambio se haya realizado correctamente

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Schemas' tree with 'transactions' expanded. The central pane contains a SQL script:

```
103
104 #Realización del cambio solicitado
105 UPDATE credit_card
106 SET iban = "TR323456312213576817699999"
107 WHERE id = "CcU-2938";
108
109 #Realizamos una verificación de que el cambio se haya realizado correctamente
110 SELECT id, iban
111 FROM credit_card
112 WHERE id = 'CcU-2938';
```

The 'Result Grid' shows one row of data:

id	iban
CcU-2938	TR323456312213576817699999

The bottom pane shows the 'Output' window with the following messages:

#	Time	Action	Message	Duration / Fetch
1	11:03:55	UPDATE credit_card SET iban = "TR323456312213576817699999" WHERE id = "CcU-2938"	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
2	11:03:59	SELECT id, iban FROM credit_card WHERE id = 'CcU-2938' LIMIT 0.500	1 row(s) returned	0.000 sec / 0.000 sec

Ejercicio 6

En la tabla "transaction" ingresa una nueva transacción con la siguiente información:

Id: 108B1D1D-5B23-A76C-55EF-C568E49A99DD,

credit_card_id: CcU-9999,

company_id: b-9999,

user_id: 9999,

lat: 829.999,

longitude: -117.999,

amount: 111.11,

declined: 0

Debido a que las tablas están relacionadas, antes de ingresar la información a la tabla **transaction**, primero debo ingresar datos a las tablas **credit_card** y **comapany**.

The screenshot shows a SQL query editor with the following commands:

```
/*Ejercicio 6
En la tabla "transaction" ingresa una nueva transacción con la siguiente información:*/
/*Id, 108B1D1D-5B23-A76C-55EF-C568E49A99DD
credit_card_id, CcU-9999
company_id, b-9999
user_id, 9999
lat, 829.999
longitude, -117.999
amount, 111.11
declined, 0 */

INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date)
VALUES ("CcU-9999", "XX00000000000000000000000000000000", "1111222233334444", "1234", "999", "12/30/36");

INSERT INTO company (id, company_name, phone, email, country, website)
VALUES ("b-9999", "Jabones lux", "6000000000", "jaboneslux@jaboneslux.com", "Spain", "www.jaboneslux.com");
```

The Output window shows the results of the execution:

#	Time	Action	Message	Duration / Fetch
1	11:07:05	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ("CcU-9999", "XX00000000000000000000000000000000", "1111222233334444", "1234", "999", "12/30/36");	1 row(s) affected 0.015 sec	
2	11:07:15	INSERT INTO company (id, company_name, phone, email, country, website) VALUES ("b-9999", "Jabones lux", "6000000000", "jaboneslux@jaboneslux.com", "Spain", "www.jaboneslux.com");	1 row(s) affected 0.000 sec	

Y ahora procedemos a ingresar los valores indicados para la tabla transaction y verificamos que se hayan introducido correctamente

The screenshot shows a SQL query editor with the following commands:

```
# Ahora procedemos a ingresar los valores indicados para la tabla transaction y verificamos que se hayan introducido correctamente
INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
VALUES ("108B1D1D-5B23-A76C-55EF-C568E49A99DD", "CcU-9999", "b-9999", "9999", 829.999, -117.999, 111.11, 0 );

SELECT id, credit_card_id, company_id, user_id, lat, longitude, amount, declined
FROM transaction
WHERE id = '108B1D1D-5B23-A76C-55EF-C568E49A99DD';
```

The Result Grid shows the data inserted into the transaction table:

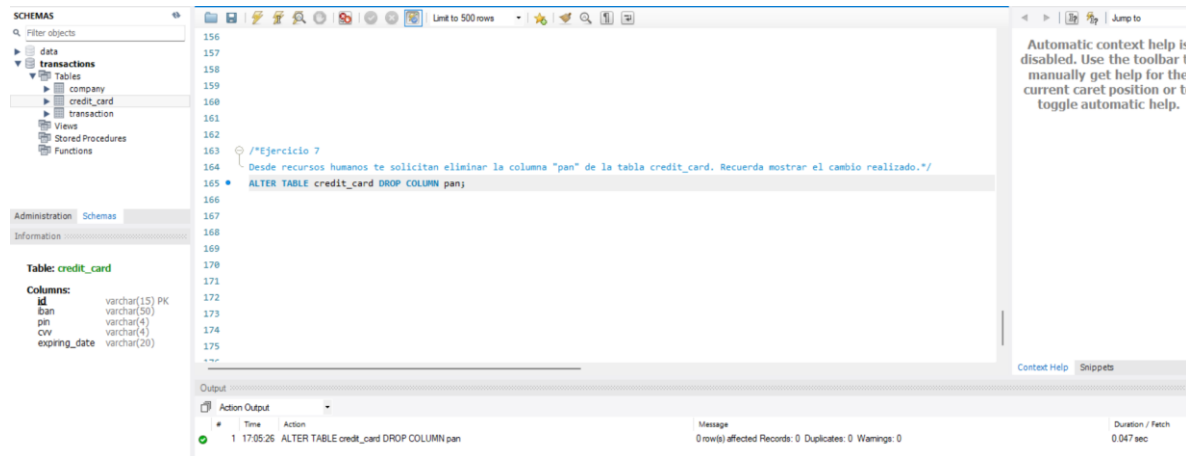
id	credit_card_id	company_id	user_id	lat	longitude	amount	declined
108B1D1D-5B23-A76C-55EF-C568E49A99DD	CcU-9999	b-9999	9999	829.999	-117.999	111.11	0

The Action Output window shows the results of the execution:

#	Time	Action	Message	Duration / Fetch
1	11:11:42	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) VALUES ("108B1D1D-5B23-A76C-55EF-C568E49A99DD", "CcU-9999", "b-9999", "9999", 829.999, -117.999, 111.11, 0);	1 row(s) affected 0.015 sec	
2	11:11:50	SELECT id, credit_card_id, company_id, user_id, lat, longitude, amount, declined FROM transaction WHERE id = '108B1D1D-5B23-A76C-55EF-C568E49A99DD';	1 row(s) returned 0.000 sec / 0.000 sec	

Ejercicio 7

Desde recursos humanos te solicitan eliminar la columna "pan" de la tabla credit_card. Recuerda mostrar el cambio realizado.



Ejercicio 8

Descarga los archivos CSV que encontrarás en el apartado de recursos :

- american_users.csv
- european_users.csv
- companies.csv
- credit_cards.csv
- transactions.csv

Estudia y diseña una base de datos con un esquema de estrella que contenga, al menos 4 tablas de las que puedas realizar las siguientes consultas: La tabla de products.csv la utilizaremos más adelante.

Creación base de datos data:

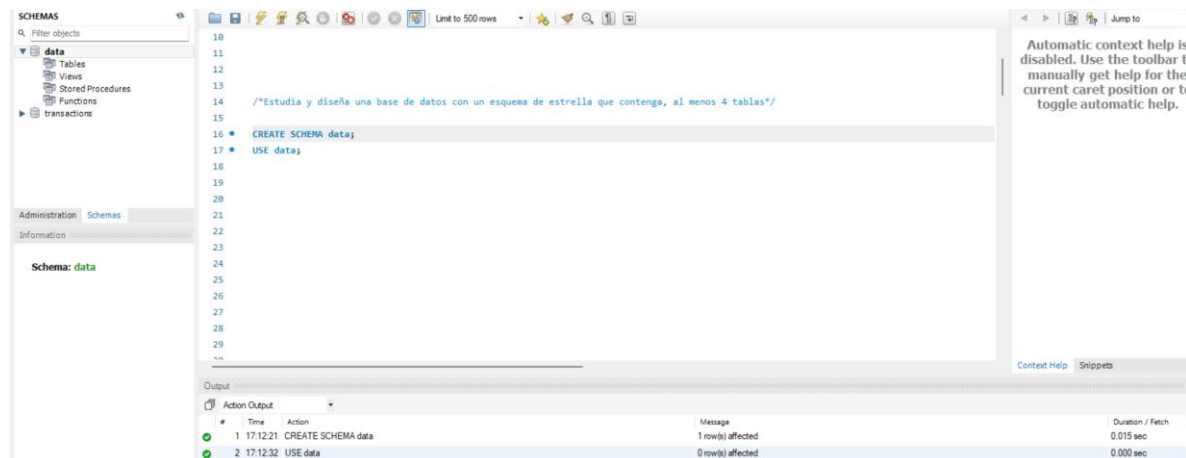


Tabla users: debido a que los archivos *american_users.csv* y *european_users.csv* contenían la misma estructura en cuanto la lista de columnas y los valores, se crea la columna *continent* para que en la tabla se logre discriminar a los usuarios por continente

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Schemas' pane shows the 'data' schema containing the 'users' table. The 'Table: users' is selected, and its columns are listed: id (PK), name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band, and continent. The main pane displays the SQL script for creating the table:

```

161
162 CREATE TABLE IF NOT EXISTS users(
163     id VARCHAR(15) PRIMARY KEY,
164     name VARCHAR(255),
165     surname VARCHAR(255),
166     phone VARCHAR(100),
167     email VARCHAR(100),
168     birth_date VARCHAR(100),
169     country VARCHAR(255),
170     city VARCHAR(255),
171     postal_code VARCHAR(100),
172     address VARCHAR(100),
173     signup_date VARCHAR(100),
174     user_segment VARCHAR(255),
175     income_band VARCHAR(255),
176     continent VARCHAR(255)
177 );
178
179

```

The Output pane at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	12:09:43	CREATE TABLE IF NOT EXISTS users(id VARCHAR(15) PRIMARY KEY, name VARCHAR(255), surname VAR...	0 row(s) affected	0.031 sec

Carga de datos tabla users: y se realiza la carga en la misma tabla

The screenshot shows the SQL Server Enterprise Manager interface. The main pane displays the SQL script for loading data into the 'users' table from two CSV files:

```

179
180 LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex-8_american_users.csv'
181 INTO TABLE users
182 FIELDS TERMINATED BY ','
183 ENCLOSED BY '"'
184 LINES TERMINATED BY '\n'
185 IGNORE 1 ROWS (id,name,surname,phone,email,birth_date,country,city,postal_code,address,signup_date,user_segment,income_band)
186 SET continent = 'american';
187
188 LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex-8_european_users.csv'
189 INTO TABLE users
190 FIELDS TERMINATED BY ','
191 ENCLOSED BY '"'
192 LINES TERMINATED BY '\n'
193 IGNORE 1 ROWS (id,name,surname,phone,email,birth_date,country,city,postal_code,address,signup_date,user_segment,income_band)
194 SET continent = 'european';
195
196
197

```

The Output pane at the bottom shows the execution results for two statements:

#	Time	Action	Message	Duration / Fetch
1	12:10:57	LOAD DATA INFILE C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex-8_american_users.csv INTO...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.047 sec
2	12:11:02	LOAD DATA INFILE C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex-8_european_users.csv INTO...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.156 sec

Tabla companies:

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Schemas' pane shows the 'data' schema containing the 'companies' table. The 'Table: companies' is selected, and its columns are listed: company_id (PK), company_name, phone, email, country, website, merchant_category, and merchant_price_position. The main pane displays the SQL script for creating the table:

```

196
197
198
199 CREATE TABLE IF NOT EXISTS companies(
200     company_id VARCHAR(15) PRIMARY KEY,
201     company_name VARCHAR(255),
202     phone VARCHAR(100),
203     email VARCHAR(100),
204     country VARCHAR(255),
205     website VARCHAR(255),
206     merchant_category VARCHAR(255),
207     merchant_price_position VARCHAR(255)
208 );
209
210

```

The Output pane at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	12:12:35	CREATE TABLE IF NOT EXISTS companies(company_id VARCHAR(15) PRIMARY KEY, company_name VAR...	0 row(s) affected	0.032 sec

Carga de datos tabla companies:

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Schemas' pane is expanded to show the 'data' database, with the 'companies' table selected. The 'Table: companies' properties window is open, showing columns: company_id (PK), company_name, phone, email, country, website, merchant_category, and merchant_price_position. The main pane displays the following SQL command:

```
211
212
213
214
215
216
217
218 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex_8_companies.csv'
219 INTO TABLE companies
220 FIELDS TERMINATED BY ','
221 LINES TERMINATED BY '\n'
222 IGNORE 1 ROWS;
223
224
225
```

The 'Output' pane at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	12:13:55	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex_8_companies.csv' INTO TABLE companies	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.053 sec

On the right, a message box states: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help."

Tabla credit_cards:

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Schemas' pane is expanded to show the 'data' database, with the 'credit_cards' table selected. The 'Table: credit_cards' properties window is open, showing columns: id (PK), user_id, iban, pan, pin, cvv, track1, track2, expiring_date, card_type, and card_renewal_flag. The main pane displays the following SQL command:

```
224
225
226 • CREATE TABLE IF NOT EXISTS credit_cards(
227 id VARCHAR(15) PRIMARY KEY,
228 user_id VARCHAR(15),
229 iban VARCHAR(30),
230 pan VARCHAR(16),
231 pin VARCHAR(4),
232 cvv VARCHAR(4),
233 track1 VARCHAR(100),
234 track2 VARCHAR(100),
235 expiring_date VARCHAR(20),
236 card_type VARCHAR(255),
237 card_renewal_flag VARCHAR(255)
238 );
239
240
241
```

The 'Output' pane at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	12:14:53	CREATE TABLE IF NOT EXISTS credit_cards (id VARCHAR(15) PRIMARY KEY, user_id VARCHAR(15), iban VARCHAR(30), pan VARCHAR(16), pin VARCHAR(4), cvv VARCHAR(4), track1 VARCHAR(100), track2 VARCHAR(100), expiring_date VARCHAR(20), card_type VARCHAR(255), card_renewal_flag VARCHAR(255))	0 row(s) affected	0.016 sec

On the right, a message box states: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help."

Carga de datos tabla credit_cards:

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Schemas' pane is expanded to show the 'data' database, with the 'credit_cards' table selected. The 'Table: credit_cards' properties window is open, showing columns: id (PK), user_id, iban, pan, pin, cvv, track1, track2, expiring_date, card_type, and card_renewal_flag. The main pane displays the following SQL command:

```
240
241
242
243
244
245
246 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex_8_credit_cards.csv'
247 INTO TABLE credit_cards
248 FIELDS TERMINATED BY ','
249 LINES TERMINATED BY '\n'
250 IGNORE 1 ROWS;
251
252
253
254
255
256
257
```

The 'Output' pane at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	12:15:38	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex_8_credit_cards.csv' INTO TABLE credit_cards	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0	0.141 sec

On the right, a message box states: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help."

Tabla transactions:

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Schemas' pane shows the 'data' schema with tables 'companies', 'credit_cards', 'transactions', 'users', 'Views', 'Stored Procedures', and 'Functions'. The 'transactions' table is selected, and its columns are listed: id (varchar(500), PK), card_id (varchar(500)), business_id (varchar(15)), timestamp (varchar(500)), amount (decimal(10,2)), declined (tinyint(1)), product_id (varchar(255)), user_id (varchar(15)), lat (float), longitude (float), discount_amount (float), tax_amount (decimal(10,2)), shipping_amount (decimal(10,2)), and channel (varchar(255)).

The main pane shows the SQL script for creating the table:

```
CREATE TABLE IF NOT EXISTS transactions(
    id VARCHAR(500) PRIMARY KEY,
    card_id VARCHAR(500),
    business_id VARCHAR(15),
    timestamp VARCHAR(500),
    amount DECIMAL(10, 2),
    declined BOOLEAN,
    product_id VARCHAR(255) REFERENCES products(id),
    user_id VARCHAR(15),
    lat FLOAT,
    longitude FLOAT,
    discount_amount FLOAT,
    tax_amount DECIMAL(10, 2),
    shipping_amount DECIMAL(10, 2),
    channel VARCHAR(255),
    campaign_id VARCHAR(15),
    device_type VARCHAR(255),
    is_international BOOLEAN,
    decline_reason VARCHAR(255),
    distance_m DECIMAL(10, 2),
    FOREIGN KEY (card_id) REFERENCES credit_cards(id),
    FOREIGN KEY (business_id) REFERENCES companies(company_id),
    FOREIGN KEY (user_id) REFERENCES users(id)
);
```

The 'Output' pane shows the execution result:

#	Time	Action	Message	Duration / Fetch
1	17:19:32	CREATE TABLE IF NOT EXISTS transactions(id VARCHAR(500) PRIMARY KEY, card_id VARCHAR(500), ...	0 row(s) affected	0.047 sec

Carga de datos tabla transactions:

The screenshot shows the SQL Server Enterprise Manager interface. The 'transactions' table is selected in the 'Schemas' pane. The main pane shows the SQL script for loading data:

```
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex-8_transactions.csv'
INTO TABLE transactions
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

The 'Output' pane shows the execution result:

#	Time	Action	Message	Duration / Fetch
1	17:31:31	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex-8_transactions.csv' INTO...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0	5.859 sec

Ejercicio 9

Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.

The screenshot shows the SQL Server Enterprise Manager interface. The 'transactions' table is selected in the 'Schemas' pane. The main pane shows the SQL script for Exercise 9:

```
/*Ejercicio 9
Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.*/
SELECT id, name, surname
FROM users u
INNER JOIN (SELECT user_id, declined
FROM transactions
GROUP BY user_id, declined
HAVING COUNT(*) > 80) AS t
ON u.id = t.user_id
WHERE t.declined = 0;
```

The 'Result Grid' shows the results of the query:

id	name	surname
185	Molly	Gillan
289	Dovigi	Hwou
318	Bnyr	Asturi
454	Sfzzoh	Xgrfndix

The 'Output' pane shows the execution result:

#	Time	Action	Message	Duration / Fetch
1	12:27:54	SELECT id, name, surname FROM users u INNER JOIN (SELECT user_id, declined FROM transactions GROU...	4 row(s) returned	0.235 sec / 0.000 sec

Ejercicio 10

Muestra la media de amount por IBAN de las tarjetas de crédito en la compañía Donec Ltd., utiliza por lo menos 2 tablas.

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

```
281 /*Ejercicio 10
282 Muestra la media de amount por IBAN de las tarjetas de crédito en la compañía Donec Ltd., utiliza por lo menos 2 tablas.*/
283 SELECT cc.iban, AVG(t.amount) AS media_amount, (SELECT company_name
284 FROM companies
285 WHERE company_name = 'Donec Ltd'
286 ) AS companies
287 FROM transactions t
288 JOIN credit_cards cc ON t.card_id = cc.id
289 WHERE t.business_id IN
290 (SELECT company_id
291 FROM companies
292 WHERE company_name = 'Donec Ltd'
293 )
294 GROUP BY cc.iban
295 ORDER BY media_amount;
```

iban	media_amount	companies
US161878381089767089885313	10.260000	Donec Ltd
US829430002334344023057024	10.620000	Donec Ltd
US4353006337810061238919	12.940000	Donec Ltd
CA79622693188512489219768	13.910000	Donec Ltd
US8459804483225006482362	14.940000	Donec Ltd
CA344265014403884610125216	15.190000	Donec Ltd
CA73408770761836747301749	19.130000	Donec Ltd

Result 7: 371 row(s) returned

Nivel 2

Ejercicio 1

Identifica los cinco días que se generó la mayor cantidad de ingresos en la empresa por ventas. Muestra la fecha de cada transacción junto con el total de las ventas.

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

```
296
297 /*Ejercicio 1
298 Identifica los cinco días que se generó la mayor cantidad de ingresos en la empresa por ventas.
299 Muestra la fecha de cada transacción junto con el total de las ventas.*/
300 SELECT DATE(timestamp) AS fecha, SUM(amount) AS total_ventas
301 FROM transactions
302 GROUP BY DATE(timestamp)
303 ORDER BY total_ventas DESC
304 LIMIT 5;
```

fecha	total_ventas
2019-11-18	15840.24
2023-02-20	15728.40
2022-12-13	15129.20
2019-03-18	14896.47
2017-12-20	14367.96

Result 8: 5 row(s) returned

Ejercicio 2

Presenta el nombre, teléfono, país, fecha y amount, de aquellas empresas que realizaron transacciones con un valor comprendido entre 350 y 400 euros y en alguna de estas fechas: 29 de abril de 2015, 20 de julio de 2018 y 13 de marzo de 2024. Ordena los resultados de mayor a menor cantidad.

```
311 /* Ejercicio 2
312 Presenta el nombre, teléfono, país, fecha y amount,
313 de aquellas empresas que realizaron transacciones con un valor comprendido entre 350 y 400 euros
314 y en alguna de estas fechas: 29 de abril de 2015, 20 de julio de 2018 y 13 de marzo de 2024.
315 Ordena los resultados de mayor a menor cantidad.*/
316 SELECT c.company_name, c.phone, c.country, DATE(t.timestamp) AS fecha, t.amount
317 FROM companies c
318 INNER JOIN transactions t
319 ON c.company_id = t.business_id
320 WHERE t.amount BETWEEN 350 AND 400
321 AND DATE(t.timestamp) IN ('2015-04-29', '2018-07-20', '2024-03-13')
322 ORDER BY t.amount DESC;
323
```

company_name	phone	country	fecha	amount
Fringilla LLC	+1 150 784 1716	New Zealand	2015-04-29	390.33
Auctor Mauris Vel LLP	+32 233 947 8239	United States	2024-03-13	383.58
Mauris Institute	+75 473 619 9472	Sweden	2015-04-29	369.20
Pede Cum Ltd	+26 625 798 6126	Norway	2018-07-20	358.64
Nelus Et Malesuada Ltd	+52 818 402 1638	Netherlands	2024-03-13	352.21
Tempor Diam Institute	+89 848 141 3136	Netherlands	2024-03-13	350.71

Result 9 x

Output

Action Output

Time Action Message Duration / Fetch

1 12:34:12 SELECT c.company_name, c.phone, c.country, DATE(t.timestamp) AS fecha, t.amount FROM companies c INVL... 6 row(s) returned 0.297 sec / 0.000 sec

Ejercicio 3

Necesitamos optimizar la asignación de los recursos y dependerá de la capacidad operativa que se requiera, por lo que te piden la información sobre la cantidad de transacciones que realizan las empresas, pero el departamento de recursos humanos es exigente y quiere un listado de las empresas en las que especifiques si tienen igual o más de 400 transacciones o menos.

```
325 /*Ejercicio 3
326 Necesitamos optimizar la asignación de los recursos y dependerá de la capacidad operativa que se requiera,
327 por lo que te piden la información sobre la cantidad de transacciones que realizan las empresas,
328 pero el departamento de recursos humanos es exigente y quiere un listado de las empresas
329 en las que especifiques si tienen igual o más de 400 transacciones o menos.*/
330 SELECT c.company_name, COUNT(*) AS cant_transacciones,
331 CASE WHEN COUNT(*) >= 400 THEN 'Igual o más de 400'
332 ELSE 'Menos de 400'
333 END AS categoria_transacciones
334 FROM companies c
335 JOIN transactions t
336 ON c.company_id = t.business_id
337 GROUP BY c.company_id, c.company_name
338 ORDER BY cant_transacciones;

```

company_name	cant_transacciones	categoria_transacciones
Lorem Eu Incorporated	380	Menos de 400
Fringilla LLC	397	Menos de 400
Nec Luctus LLC	399	Menos de 400
Dui Quis Institute	402	Igual o más de 400
Raus Associates	407	Igual o más de 400
Magna A Neque Industries	410	Igual o más de 400

Result 10 x

Output

Action Output

Time Action Message Duration / Fetch

1 12:35:54 SELECT c.company_name, COUNT(*) AS cant_transacciones, CASE WHEN COUNT(*) >= 400 THEN 'Igual o ... 100 row(s) returned 0.250 sec / 0.000 sec

Ejercicio 4

Elimina de la tabla transacción el registro con ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de datos.

Verificamos que exista el ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD en la tabla transacciones

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Schemas' tree with 'data' selected. The central pane shows a query editor with the following SQL script:

```
/*Ejercicio 4
Elimina de la tabla transacción el registro con ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de datos.*/

#Verificamos que exista el ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD en la tabla transacciones
SELECT id
FROM transactions
WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

The 'Result Grid' shows one row with the ID '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'. The 'Output' pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	12:39:35	SELECT id FROM transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD' LIMIT 0, 500	1 row(s) returned	0.015 sec / 0.000 sec

Se elimina el registro y realizamos una verificación de que el cambio se haya realizado correctamente

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Schemas' tree with 'data' selected. The central pane shows a query editor with the following SQL script:

```
#Se elimina el registro y realizamos una verificación de que el cambio se haya realizado correctamente
DELETE FROM transactions
WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';

SELECT id
FROM transactions
WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

The 'Result Grid' shows one row with the ID '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'. The 'Output' pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	12:41:25	DELETE FROM transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) affected	0.031 sec
2	12:42:28	SELECT id FROM transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD' LIMIT 0, 500	0 row(s) returned	0.000 sec / 0.000 sec

Ejercicio 5

La sección de marketing desea tener acceso a información específica para realizar análisis y estrategias efectivas. Se ha solicitado crear una vista que proporcione detalles clave sobre las compañías y sus transacciones. Será necesaria que crees una vista llamada VistaMarketing que contenga la siguiente información: Nombre de la compañía. Teléfono de contacto. País de residencia. Media de compra realizado por cada compañía.

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Schemas' tree with 'data' expanded, showing 'Tables' (companies, credit_cards, transactions, users) and 'Views' (vistamarketing). The central pane contains the following SQL script:

```
358
359 /*Ejercicio 5
360 La sección de marketing desea tener acceso a información específica para realizar análisis y estrategias efectivas.
361 Se ha solicitado crear una vista que proporcione detalles clave sobre las compañías y sus transacciones.
362 Será necesaria que crees una vista llamada VistaMarketing que contenga la siguiente información:
363 Nombre de la compañía. Teléfono de contacto. País de residencia. Media de compra realizado por cada compañía.*/
364 CREATE VIEW VistaMarketing AS
365 SELECT c.company_name, c.phone, c.country, AVG(t.amount) AS med_compras
366 FROM companies c
367 JOIN transactions t
368 ON c.company_id = t.business_id
369 GROUP BY c.company_id, c.company_name, c.phone, c.country
370 ORDER BY med_compras DESC;
371
```

The right pane shows a message: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help." The bottom pane shows the 'Output' window with the following message:

#	Time	Action	Message	Duration / Fetch
1	12:46:24	CREATE VIEW VistaMarketing AS SELECT c.company_name, c.phone, c.country, AVG(t.amount) AS med_compras	0 row(s) affected	0.000 sec

Presenta la vista creada, ordenando los datos de mayor a menor promedio de compra.

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Schemas' tree with 'data' expanded, showing 'Tables' (companies, credit_cards, transactions, users) and 'Views' (vistamarketing). The central pane contains the following SQL script:

```
377
378
379
380 /*Presenta la vista creada, ordenando los datos de mayor a menor promedio de compra.*/
381 SELECT * FROM VistaMarketing;
382
383
```

The right pane shows a message: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help." The bottom pane shows the 'Output' window with the following message:

#	Time	Action	Message	Duration / Fetch
1	12:47:14	SELECT * FROM VistaMarketing LIMIT 0, 500	100 row(s) returned	0.567 sec / 0.000 sec

The 'Result Grid' shows the following data:

company_name	phone	country	med_compras
Ac Fermentum Incorporated	+64 601 244 6320	Germany	330.769338
Ut Semper Foundation	+10 283 500 6449	Sweden	307.372945
Oro Adipiscing Limited	+40 897 390 5820	United Kingdom	306.265648
Dolor Vitae Limited	+46 421 367 9195	France	305.952745
Nunc In Foundation	+13 267 741 2619	Italy	305.396141

Nivel 3

Ejercicio 1

Crea una nueva tabla que refleje el estado de las tarjetas de crédito basado en si las tres últimas transacciones han sido declinadas entonces es inactivo, si al menos una no es rechazada entonces es activo.

The screenshot shows a database IDE with a schema tree on the left. The 'data' schema contains tables: companies, credit_cards, estado_tarjetas, transactions, and users. The 'estado_tarjetas' table is highlighted, showing its columns: card_id (varchar(500)) and estado (varchar(8)). The main editor displays the following SQL code:

```
369
370 #Nivel 3
371 • USE data;
372 /*Ejercicio 1
373 Crea una nueva tabla que refleje el estado de las tarjetas de crédito basado en si las tres últimas transacciones han sido declinadas
374 entonces es inactivo, si al menos una no es rechazada entonces es activo. Partiendo de esta tabla responde:*/
375 • CREATE TABLE estado_tarjetas AS
376 SELECT card_id,
377 CASE WHEN SUM(CASE WHEN declined = 0 THEN 1 ELSE 0 END) >= 1
378 THEN 'Activa' ELSE 'Inactiva'
379 END AS estado
380 FROM(
381 SELECT card_id, declined, timestamp,
382 ROW_NUMBER() OVER ( PARTITION BY card_id
383 ORDER BY timestamp DESC
384 ) AS rn
385 FROM transactions
386 ) ultimas
387 WHERE ultimas.rn <= 3
388 GROUP BY card_id;
```

The output pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	13:11:58	CREATE TABLE estado_tarjetas AS SELECT card_id, CASE WHEN SUM(CASE WHEN declined = 0 THEN 1 ELSE 0 END) >= 1 THEN 'Activa' ELSE 'Inactiva' END AS estado FROM transactions	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.438 sec

Partiendo de esta tabla responde:

¿Cuántas tarjetas están activas?

The screenshot shows the same database IDE with the 'estado_tarjetas' table selected in the schema tree. The main editor displays the following SQL code:

```
394
395
396
397
398 #¿Cuántas tarjetas están activas?
399 • SELECT COUNT(*) AS cant_tarjetas, estado
400 FROM estado_tarjetas
401 WHERE estado = 'Activa'
402 group by estado;
```

The output pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	13:12:46	SELECT COUNT(*) AS cant_tarjetas, estado FROM estado_tarjetas WHERE estado = 'Activa' group by estado	1 row(s) returned	0.016 sec / 0.000 sec

Ejercicio 2:

Crea una tabla con la que podamos unir los datos del archivo de products.csv con la base de datos creada (ya que hasta ahora no podíamos hacerlo), teniendo en cuenta que desde transaction tienes product_ids.

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Schemas' pane shows the 'data' schema with tables like 'companies', 'credit_cards', 'estado_tarjetas', 'products', 'transactions', 'transact_products', and 'users'. The 'transact_products' table is highlighted. The main pane shows the SQL script for creating the table:

```
42
43
44 /*Ejercicio 2
45 Crea una tabla con la que podamos unir los datos del archivo de products.csv con la base de datos creada
46 (ya que hasta ahora no podíamos hacerlo), teniendo en cuenta que desde transaction tienes product_ids. */
47 CREATE TABLE transact_products AS
48 SELECT t.id AS transaction_id, jt.product_id
49 FROM transactions t
50 JOIN JSON_TABLE(
51   CONCAT('["', REPLACE(t.product_ids, ',', "','"), '"]'), '$[*]'
52   ) AS jt;
53
```

The 'Output' pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	17:15:19	CREATE TABLE transact_products AS SELECT t.id AS transaction_id, jt.product_id FROM transactions t JOIN JS...	253388 row(s) affected. Records: 253388 Duplicates: 0 Warnings: 0	2.594 sec

creación tabla products:

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Schemas' pane shows the 'data' schema with tables like 'companies', 'credit_cards', 'estado_tarjetas', 'products', 'transactions', 'transact_products', and 'users'. The 'products' table is highlighted. The main pane shows the SQL script for creating the table:

```
54
55
56 /* creacion tabla products*/
57 CREATE TABLE IF NOT EXISTS products(
58   id VARCHAR(15) PRIMARY KEY,
59   product_name VARCHAR(255),
60   price VARCHAR(100),
61   colour VARCHAR(100),
62   weight VARCHAR(255),
63   warehouse_id VARCHAR(255),
64   category VARCHAR(255),
65   brand VARCHAR(255),
66   cost VARCHAR(255),
67   launch_date VARCHAR(255)
68 );

```

The 'Output' pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	17:17:37	CREATE TABLE IF NOT EXISTS products(id VARCHAR(15) PRIMARY KEY, product_name VARCHAR(255), pr...	0 row(s) affected	0.031 sec

Carga datos tabla products:

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Schemas' pane shows the 'data' schema with tables like 'companies', 'credit_cards', 'estado_tarjetas', 'products', 'transactions', 'transact_products', and 'users'. The 'products' table is highlighted. The main pane shows the SQL script for loading data into the table:

```
69
70
71
72
73
74 /* Carga datos tabla products*/
75 LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex-8_products.csv'
76 INTO TABLE products
77 FIELDS TERMINATED BY ','
78 LINES TERMINATED BY '\n'
79 IGNORE 1 ROWS;
80
81

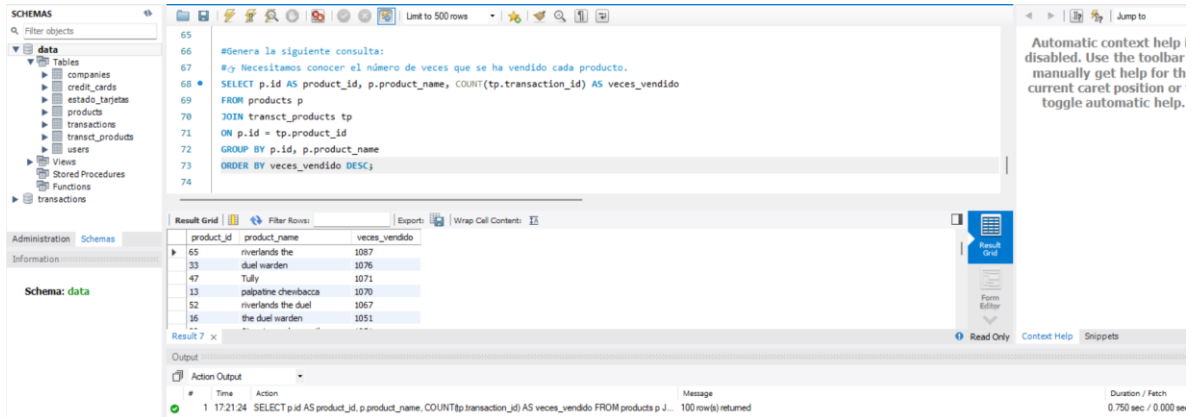
```

The 'Output' pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	17:18:48	LOAD DATA INFILE C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex-8_products.csv INTO TABL...	100 row(s) affected. Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.062 sec

Genera la siguiente consulta:

Necesitamos conocer el número de veces que se ha vendido cada producto.



The screenshot shows a database management tool interface. On the left, a 'SCHEMAS' panel lists various tables including 'companies', 'credit_cards', 'estado_targetes', 'products', 'transactions', 'transact_products', 'users', 'Views', 'Stored Procedures', 'Functions', and 'transactions'. The main editor displays a SQL query to count the number of times each product has been sold. The query is as follows:

```
65
66 #Genera la siguiente consulta:
67 #.y Necesitamos conocer el número de veces que se ha vendido cada producto.
68 • SELECT p.id AS product_id, p.product_name, COUNT(tp.transaction_id) AS veces_vendido
69 FROM products p
70 JOIN transact_products tp
71 ON p.id = tp.product_id
72 GROUP BY p.id, p.product_name
73 ORDER BY veces_vendido DESC;
74
```

Below the query, the 'Result Grid' shows the following data:

product_id	product_name	veces_vendido
65	riverlands the	1087
33	duel warden	1076
47	Tully	1071
13	palpatine chewbacca	1070
52	riverlands the duel	1067
16	the duel warden	1051

At the bottom, the 'Output' panel shows the execution details: 'Action Output' for the query, 'Time' of 17:21:24, 'Message' indicating 100 row(s) returned, and 'Duration / Fetch' of 0.750 sec / 0.000 sec.